

上下文无关文法与下推自动机

CFG 化简与 PDA→CFG 转换实验报告

Formal Languages Lab 2 • CFG Simplification & PDA to CFG

小组成员

张恒基（组长）

2024211301

2024210926

尹浩铭

2024211301

2024210910

林旭东

2024211301

2024210915

赵博宇

2024211301

2024210908

项目代号: formal-language-lab2

2026 年 5 月 22 日

目录

小组信息与实验环境	1
成员分工	1
实验环境	1
项目结构	1
程序设计	1
整体架构	1
命令行接口	2
CFG 化简流水线	2
PDA 到 CFG 组合流水线	3
核心算法	3
epsilon 产生式消除	3
单产生式消除	3
无用符号消除	3
消除非生成符号	3
消除不可达符号	4
PDA 到 CFG 等价构造	4
输入与输出格式	4
CFG 输入	4
PDA 输入	4
输出格式	5
测试用例与执行效果	5
CFG 指定样例	5
PDA 指定样例	6
自动化测试	7
可执行程序与打包	7
改进思路	8
附录 A: 命令快速参考	8
附录 B: 版本记录	8

小组信息与实验环境

成员分工

姓名	学号	角色	分工与负责模块
张恒基	2024210926	组长	CFG 数据结构与化简算法; cfg.py、cfg_parser.py、cfg_simplifier.py; epsilon / 单产生式 / 无用符号消除及算法文档。
林旭东	2024210915	PDA / 转换	PDA 数据结构与 PDA→CFG; pda.py、pda_parser.py、pda_to_cfg.py; [p,A,q] 变量构造。
尹浩铭	2024210910	入口 / 测试	命令行与样例测试; main.py、examples/、tests/; 12 项单元测试与运行记录。
赵博宇	2024210908	文档 / 演示	实验报告、可执行说明与演示脚本; README.md、docs/、PyInstaller 打包与视频脚本。

实验环境

项目	配置
操作系统	Windows 11 (x86-64)
编程语言	Python 3.12 (兼容 3.9+)
依赖	仅标准库: dataclasses、itertools、argparse、re、unittest
运行入口	main.py; 可选 dist/formal_lang_lab2.exe
打包	PyInstaller --onefile --name formal_lang_lab2

项目结构

```
仓库目录 (提交源码根目录)

formal-language-lab2/
├─ main.py           # 命令行入口 (simplify / pda2cfg / demo)
├─ cfg.py            # CFG 数据结构
├─ cfg_parser.py     # CFG 文本解析
├─ cfg_simplifier.py # CFG 化简 (四步流水线)
├─ pda.py           # PDA 五元组
├─ pda_parser.py     # PDA 文本解析 (块格式 + 数学符号格式)
├─ pda_to_cfg.py     # PDA→CFG 标准构造
├─ examples/
│   ├── grammar_sample.txt # 指定 CFG 样例
│   ├── specified_cfg.txt
│   ├── pda_sample.txt    # 指定 PDA 样例
│   └── specified_pda.txt
├─ tests/           # 12 项 unittest
└─ docs/           # 本报告、截图、可执行说明
```

程序设计

整体架构

项目采用模块化分层: 数据结构层 (cfg.py、pda.py) → 解析层 (cfg_parser.py、pda_parser.py) → 算法层 (cfg_simplifier.py、pda_to_cfg.py), 由 main.py 统一调度。

1. 算法与数据解耦：化简只操作 CFG 对象；PDA 转换只操作 PDA 对象，与文本格式无关。可组合：pda2cfg 输出 CFG 后可接 simplify_cfg，满足「先转换再化简」。可测试：各模块独立单测，不依赖命令行或磁盘 I/O（除解析入口）。

源文件	职责
cfg.py	产生式集合、变量/终结符集合、文本输出
cfg_parser.py	多箭头写法、eps 别名、注释过滤
cfg_simplifier.py	nullable / unit-closure / 生成 / 可达四步化简
pda.py	状态、栈符号、迁移表、空栈接受标志
pda_parser.py	块格式与 $\delta(q,a,A)=$ 数学格式
pda_to_cfg.py	$[p,A,q]$ 变量法、压栈长度枚举
main.py	子命令 simplify、pda2cfg、demo

Table 1: 模块划分与源文件对应关系

命令行接口

子命令	别名	功能
simplify	simplify-cfg	读入 CFG 文件，输出化简后产生式
pda2cfg	pda-to-cfg	读入 PDA，输出转换 CFG；--simplify 继续化简
demo	—	内置 SPECIFIED_CFG 与 SPECIFIED_PDA 一次演示

处理流水线：读文件 → 解析 → 算法 → 标准产生式文本输出。

CFG 化简流水线

```

输入 CFG 文本
    |
    v
  cfg_parser.parse_cfg()
    |
    v
(1) eliminate_epsilon_productions
    |
    v
(2) eliminate_unit_productions
    |
    v
(3) remove_non_generating_symbols
    |
    v
(4) remove_unreachable_symbols
    |
    v
  CFG.to_text()  --> 终端输出
  
```

Listing 1: CFG 化简四步流水线（顺序不可调换）

顺序不可调换：消 epsilon 可能引入新单产生式；消单产生式可能产生新的非生成符号；先删非生成再删不可达，才能保留「可生成终结符串且从开始符号可达」的符号。

PDA 到 CFG 组合流水线

```
输入 PDA 文本
  |
  v
pda_parser.parse_pda()
  |
  v
pda_to_cfg(pda) --> 原始 CFG (含 [p,A,q] 变量)
  |
  +---- 默认: 直接打印
  |
  +---- --simplify: simplify_cfg() --> 化简 CFG
  |
  v
终端输出产生式列表
```

Listing 2: PDA→CFG 与可选化简组合流程

核心算法

epsilon 产生式消除

问题: $A \rightarrow \varepsilon$ 使语言可能含空串; 目标是在语言不含空串时得到无 **epsilon** 产生式的等价文法。

步骤:

1. 计算 **nullable** (不动点): 若 $A \rightarrow \alpha$ 且 α 中符号均在 nullable, 则 $A \in \text{nullable}$ 。扩展右部: 对每个产生式, 枚举 nullable 位置的所有删除组合, 加入新产生式。删除: 原 epsilon 产生式不再保留 (空右部不加入, 除非可选保留 $S \rightarrow \varepsilon$)。

复杂度: nullable 迭代 $O(|V| \cdot |P|)$; 右部 k 个 nullable 时组合 2^k 。

实现: cfg_simplifier.py 中 find_nullable_variables()、itertools.combinations。

单产生式消除

问题: $A \rightarrow B$ 不直接产生终结符, 形成 $A \xRightarrow{*} B$ 链, 使文法冗余。

步骤:

1. 对每个 A 计算 unit_closure[A] (经单产生式可达的变量集, BFS/DFS)。将闭包中每个 B 的非单产生式并入 A 。删除全部单产生式。

复杂度: $O(|V| \cdot (|V| + |P|))$ 。

实现: _unit_reachable_variables()、_is_unit_production()。

无用符号消除

分两步, 必须先非生成、后不可达。

消除非生成符号

1. 不动点计算 generating: 产生式右部变量均在 generating 中则左部加入 (终结符恒可生成)。删除左部非 generating 的产生式; 右部含非生成变量的产生式一并删除。

消除不可达符号

1. 从 $\{S\}$ 出发 BFS 得 `reachable`。仅保留左部在 `reachable` 的产生式；右部含不可达变量的产生式删除。

实现：`find_generating_variables()`、`find_reachable_variables()`
(`cfg_simplifier.py:77-138`)。

PDA 到 CFG 等价构造

前置：仅支持空栈接受 (`accepts_by_empty_stack = True`)。

步骤	说明
1. 变量	对每个 (p, A, q) 引入 $[p, A, q]$ ：从 p 出发栈顶 A ，读串后弹出 A 到 q ；共 $ Q ^2 \Gamma $ 个
2. 开始式	$S \rightarrow [q_0, z_0, q]$, q 取遍所有状态
3. 迁移	$\delta(p, a, A) = (q, \varepsilon) \rightarrow [p, A, q] \rightarrow a$
4. 压栈	$\delta(p, a, A) = (q, B_1 \dots B_k)$ 时枚举中间状态，链式产生式
5. ε 迁移	输入 ε 时右部可为纯变量串

复杂度：压栈长度 k 时单条迁移最多 $O(|Q|^k)$ 条产生式；总 worst-case $O(|Q|^{K+1}|\Gamma|)$ 。

实现：`itertools.product` 枚举中间状态；`_add_transition_productions()` 区分 push 长度 0 与 ≥ 1 。

输入与输出格式

CFG 输入

紧凑风格：

```
S -> a | bA | B | ccD
A -> abB | epsilon
B -> aA
C -> ddC
D -> ddd
```

多字符变量 (PDA 转换后)：

```
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1]
```

规则 说明

箭头 $\rightarrow$ 、 $\Rightarrow$ 、 $\rightarrow$ 等

空串 `epsilon`、`eps`、 ε 等

分隔 `|` 分隔候选右部；`#` 起为注释

推断 开始符号、变量集、终结符集由产生式自动收集

PDA 输入

块格式 (推荐, `examples/pda_sample.txt`)：

```

states: q0 q1
input_symbols: a b
stack_symbols: B z0
start_state: q0
start_stack: z0
accept: empty_stack
transitions:
q0,b,z0 -> q0,B z0
q0,b,B -> q0,B B
q0,a,B -> q1,eps
q1,a,B -> q1,eps
q1,eps,B -> q1,eps
q1,eps,z0 -> q1,eps

```

数学符号格式：与 tests/test_pda_to_cfg.py 中 SPECIFIED_PDA 字符串一致；解析器归一化 δ 、 ϵ 、全角括号等。

输出格式

按变量名排序输出（ S 固定最前）；多字符符号以空格分隔；空串统一打印 epsilon。

测试用例与执行效果

CFG 指定样例

输入（examples/grammar_sample.txt）：

化简前（课程指定 CFG）

```

S -> a | b A | B | c c D
A -> a b B | eps
B -> a A
C -> d d C
D -> d d d

```

分析要点：

1. $A \rightarrow \epsilon \Rightarrow A$ nullable; $B \rightarrow aA$ 展开得 $B \rightarrow aA | a$ 。 $S \rightarrow bA$ 展开得 $S \rightarrow bA | b$; $S \rightarrow B$ 为单产生式，unit-closure 将 B 的非单产生式并入 S 。 $C \rightarrow ddC$ 仅自引用 $\Rightarrow C$ 非生成，应删除。

命令：

```
py main.py simplify examples\grammar_sample.txt
```

输出：

化简后（终端实测）

```

S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd

```

结果： C 删除； ϵ 与 $S \rightarrow B$ 消除； $D \rightarrow ddd$ 保留。

```
PS C:\projects\formal-language-lab2> py main.py simplify examples\grammar_sample.txt
S -> a | aA | b | bA | ccD
A -> abB
B -> a | aA
D -> ddd
PS C:\projects\formal-language-lab2>
```

Figure 1: CFG 化简命令运行截图

PDA 指定样例

输入 **PDA**：见 examples/pda_sample.txt；等价于 main.py 内 SPECIFIED_PDA。

识别语言：

$$L = \{b^m a^n m \mid m \geq 1, n \geq 1, n \leq m\}$$

(先读 b 压栈，再读 a/ϵ 弹栈；不是 $b^n a^n$ 。与 test_simplified_cfg_matches_specified_pda_examples 一致。)

命令：

```
py main.py pda2cfg examples\pda_sample.txt --simplify
```

输出：

转换并化简后（4 条产生式）

```
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
```

产生式含义：

产生式	对应迁移直觉
$S \rightarrow b [q_0, B, q_1]$	从 q_0 弹 z_0 且读 b 压 B 到 q_1 的路径
$[q_0, B, q_1] \rightarrow a$	$\delta(q_0, a, B) = (q_1, \epsilon)$
$[q_0, B, q_1] \rightarrow b [q_0, B, q_1]$	$\delta(q_0, b, B) = (q_0, B)$ ，中间状态 $r = q_1$
$[q_0, B, q_1] \rightarrow b [q_0, B, q_1] [q_1, B, q_1]$	同上， $r = q_0$ 形成递归
$[q_1, B, q_1] \rightarrow a$	$\delta(q_1, a, B) = (q_1, \epsilon)$

语义验证（BFS 枚举短串）：可生成 ba 、 bba 、 $bbaa$ 、 $bbbbaa$ ；不可生成 baa 、空串、单独 b/a 。


```
PS C:\projects\formal-language-lab2> py main.py pda2cfg examples\pda_sample.txt --simplify
Simplified CFG:
S -> b [q0,B,q1]
[q0,B,q1] -> a | b [q0,B,q1] | b [q0,B,q1] [q1,B,q1]
[q1,B,q1] -> a
PS C:\projects\formal-language-lab2>
```

Figure 2: PDA 转 CFG 并化简运行截图

自动化测试

测试文件	用例数	覆盖内容
test_cfg_parser.py	2	指定 CFG、多字符符号解析
test_cfg_simplifier.py	3	nullable、指定 CFG 化简、单产生式环
test_pda_parser.py	2	块格式 / 数学符号 PDA
test_pda_to_cfg.py	3	变量构造、压栈长度、化简后语言 BFS 验证
test_main.py	2	demo 输出、CLI 别名

Table 2: 单元测试覆盖（合计 12 项）

test_pda_to_cfg.py 对化简 CFG 做有界 BFS，逐串校验是否属于上述 L ，保证「转换 + 化简」端到端正确。

测试命令与结果

```
py -m unittest discover -s tests
Ran 12 tests in 0.08s - OK
```

```
PS C:\projects\formal-language-lab2> C:\Users\bos\AppData\Local\Programs\Python\Python312\python.exe -m unittest discover -s tests
.....
-----
Ran 12 tests in 0.005s

OK
PS C:\projects\formal-language-lab2>
```

Figure 3: unittest 运行截图

可执行程序与打包

步骤 命令

安装	pip install pyinstaller
打包	pyinstaller --onefile --name formal_lang_lab2 main.py
产物	dist/formal_lang_lab2.exe
验证	dist\formal_lang_lab2.exe demo

demo 子命令实测输出摘要

```
Specified CFG simplified:
S -> a | aA | b | bA | ccD
...
Specified PDA converted and simplified:
```

```
S -> b [q0,B,q1]
...
```

改进思路

方向	说明
图形界面	Web (Streamlit/Gradio) 分步展示四步化简中间文法
边界测试	全非生成文法、纯 epsilon、长单产生式链、压栈 $k > 3$
--keep-start-epsilon	语言含空串时可选保留 $S \rightarrow \epsilon$
终态接受 PDA	自动插入 ϵ 迁移转为空栈接受
--verbose	打印每步化简后的 CFG

附录 A：命令快速参考

命令	说明
py main.py simplify FILE	CFG 化简
py main.py pda2cfg FILE	PDA→CFG (原始)
py main.py pda2cfg FILE --simplify	PDA→CFG 再化简
py main.py demo	内置两例演示
py -m unittest discover -s tests	全部单元测试

附录 B：版本记录

版本	日期	说明
v1.0	2026-05	完成 CFG 化简、PDA→CFG、命令行与 12 项测试
v1.1	2026-05-22	填充运行截图；Typst 报告排版对齐 INTERFACE.typ